```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torchvision.utils import save_image, make_grid
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import numpy as np
import os
from tqdm.auto import tqdm

# --- Configuration (T4 Optimized) ---
DATASET_CHOICE = 'mnist'
EPOCHS = 50
BATCH_SIZE = 128  # Increased for T4
NOISE_DIM = 100
LR = 0.0002
SAVE_INTERVAL = 5
IMG_SIZE = 28
CHANNELS = 1

# Device Config with Optimizations
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

if device.type == 'cuda':
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"Memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")
    # Optimize for fixed input size
    torch.backends.cudnn.benchmark = True
else:
    print("⚠️  No GPU detected. Training will be slow.")

# Create directories
os.makedirs("generated_samples/01", exist_ok=True)
os.makedirs("final_generated_images/01", exist_ok=True)
os.makedirs("models/01", exist_ok=True)
print("Directories created.")
```

```
Using device: cuda
GPU: Tesla T4
Memory: 15.83 GB
Directories created.
```

```
# --- Step 1: Train a Classifier (for later evaluation) ---
class SimpleClassifier(nn.Module):
    def __init__(self):
        super(SimpleClassifier, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)
        self.conv2 = nn.Conv2d(32, 64, 3, 1)
        self.fc1 = nn.Linear(9216, 128)
        self.fc2 = nn.Linear(128, 10)
        self.pool = nn.MaxPool2d(2)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.conv1(x))
        x = self.relu(self.conv2(x))
        x = self.pool(x)
        x = torch.flatten(x, 1)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

def train_classifier():
    print("Training Evaluation Classifier (1 epoch)...")
    transform_cls = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])
    ds = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform_cls)
    # Optimized DataLoader
    dl = DataLoader(ds, batch_size=128, shuffle=True, num_workers=2, pin_memory=True)

    model = SimpleClassifier().to(device)
    crit = nn.CrossEntropyLoss()
    opt = optim.Adam(model.parameters(), lr=0.001)

    model.train()
    for imgs, lbls in tqdm(dl, desc="Training Classifier"):
        imgs, lbls = imgs.to(device), lbls.to(device)
        opt.zero_grad()
```

```
        out = model(imgs)
        loss = crit(out, lbls)
        loss.backward()
        opt.step()

    print("Classifier trained.")
    torch.save(model.state_dict(), "models/01/mnist_classifier.pth")
    return model

if not os.path.exists("models/01/mnist_classifier.pth"):
    eval_classifier = train_classifier()
else:
    print("Loading existing classifier...")
    eval_classifier = SimpleClassifier().to(device)
    eval_classifier.load_state_dict(torch.load("models/01/mnist_classifier.pth"))
    eval_classifier.eval()
```

```
Training Evaluation Classifier (1 epoch)...
100%|██████████| 9.91M/9.91M [00:00<00:00, 17.9MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 482kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.48MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 2.60MB/s]
Training Classifier: 100%                                469/469 [00:13<00:00, 31.77it/s]
Classifier trained.
```

```
# --- Step 2: GAN Setup ---

# Data Loading (Normalized to [-1, 1])
gan_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

dataset = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=gan_transform)
# Optimized DataLoader
dataloader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True, num_workers=2, pin_memory=True)

# 1. Generator
class Generator(nn.Module):
    def __init__(self, noise_dim, img_dim):
        super(Generator, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(noise_dim, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 512),
            nn.LeakyReLU(0.2),
            nn.Linear(512, 1024),
            nn.LeakyReLU(0.2),
            nn.Linear(1024, img_dim),
            nn.Tanh() # Outputs -1 to 1
        )
    def forward(self, x):
        return self.net(x)

# 2. Discriminator
class Discriminator(nn.Module):
    def __init__(self, img_dim):
        super(Discriminator, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(img_dim, 512),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),
            nn.Linear(256, 1),
            nn.Sigmoid() # Outputs 0 to 1
        )
    def forward(self, x):
        return self.net(x)

# Init Models
gen = Generator(NOISE_DIM, 28*28).to(device)
disc = Discriminator(28*28).to(device)

opt_g = optim.Adam(gen.parameters(), lr=LR)
opt_d = optim.Adam(disc.parameters(), lr=LR)
criterion = nn.BCELoss()

fixed_noise = torch.randn(25, NOISE_DIM).to(device)
```

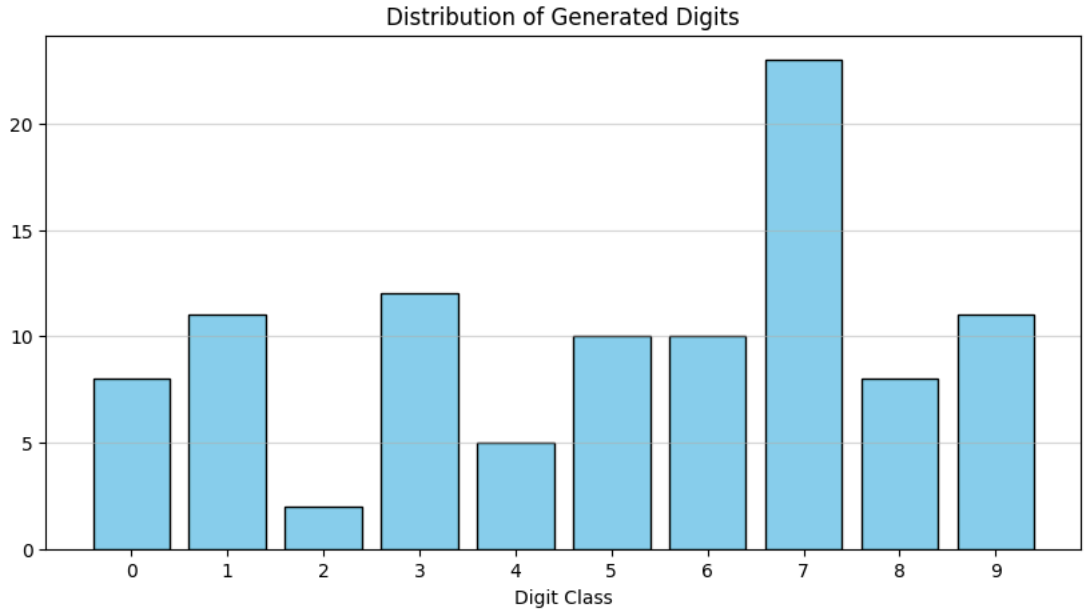
```
print("GAN Models initialized.")
```

```
GAN Models initialized.
```

```
# --- Step 3: Training Loop ---
print(f"Starting Training for {EPOCHS} epochs...")

for epoch in range(EPOCHS):
    # progress bar for the epoch
    pbar = tqdm(enumerate(dataloader), total=len(dataloader), desc=f"Epoch {epoch+1}/{EPOCHS}")

    for batch_idx, (real, _) in pbar:
        real = real.view(-1, 784).to(device)
        curr_batch_size = real.shape[0]

        real_labels = torch.ones(curr_batch_size, 1).to(device)
        fake_labels = torch.zeros(curr_batch_size, 1).to(device)

        # --- Train Discriminator ---
        outputs = disc(real)
        d_loss_real = criterion(outputs, real_labels)
        real_score = outputs.mean().item()

        noise = torch.randn(curr_batch_size, NOISE_DIM).to(device)
        fake = gen(noise)
        outputs = disc(fake.detach())
        d_loss_fake = criterion(outputs, fake_labels)
        fake_score = outputs.mean().item()

        d_loss = (d_loss_real + d_loss_fake) / 2
        opt_d.zero_grad()
        d_loss.backward()
        opt_d.step()

        # --- Train Generator ---
        outputs = disc(fake)
        g_loss = criterion(outputs, real_labels)

        opt_g.zero_grad()
        g_loss.backward()
        opt_g.step()

        # Update progress bar
        pbar.set_postfix({
            'D_loss': f"{d_loss.item():.4f}",
            'G_loss': f"{g_loss.item():.4f}",
            'Real_S': f"{real_score:.2f}",
            'Fake_S': f"{fake_score:.2f}"
        })

    # --- Epoch Logging & Saving ---
    if (epoch+1) % SAVE_INTERVAL == 0:
        with torch.no_grad():
            fake_images = gen(fixed_noise).reshape(-1, 1, 28, 28)
            save_image(fake_images, f"generated_samples/01/epoch_{epoch+1:02d}.png", nrow=5, normalize=True)
            print(f"  -> Saved sample for epoch {epoch+1}")

print("Training Complete.")
```

```
# --- Step 4: Evaluation ---
print("Generating 100 final samples...")
gen.eval()
with torch.no_grad():
    # 1. Generate
    final_noise = torch.randn(100, NOISE_DIM).to(device)
    final_fake = gen(final_noise).reshape(-1, 1, 28, 28)

    # 2. Save
    save_image(final_fake, "final_generated_images/01/final_100_samples.png", nrow=10, normalize=True)

    # 3. Predict
    eval_classifier.to(device)
    eval_classifier.eval()

    outputs = eval_classifier(final_fake)
    _, predicted = torch.max(outputs.data, 1)

    # 4. Report
    preds = predicted.cpu().numpy()

    plt.figure(figsize=(10, 5))
    plt.hist(preds, bins=range(11), align='left', rwidth=0.8, color='skyblue', edgecolor='black')
    plt.xticks(range(10))
    plt.xlabel('Digit Class')
    plt.title('Distribution of Generated Digits')
    plt.grid(axis='y', alpha=0.5)
    plt.show()

    print("Predicted labels:", preds)
    unique, counts = np.unique(preds, return_counts=True)
    print(dict(zip(unique, counts)))
```

```
Generating 100 final samples...
```




```
Predicted labels: [4 5 9 7 3 7 7 7 1 3 6 6 7 9 3 5 7 4 9 7 7 5 3 6 1 6 7 5 8 4 2 1 1 4 6 5 7
 6 6 0 7 1 4 7 0 9 3 1 5 8 8 6 0 3 7 8 3 3 1 7 5 5 7 7 7 9 3 9 7 5 8 7 6 3
 7 7 0 1 5 3 9 1 3 9 0 1 0 1 6 9 9 8 8 9 0 0 2 7 8 7]
{np.int64(0): np.int64(8), np.int64(1): np.int64(11), np.int64(2): np.int64(2), np.int64(3): np.int64(12), np.int64(4): np.i
```

```
Epoch 32/50: 100%                    468/468 [00:15<00:00, 24.30it/s, D_loss=0.5145, G_loss=1.4271, Real_S=0.64, Fake_S=0.33]
Epoch 33/50: 100%                    468/468 [00:15<00:00, 32.18it/s, D_loss=0.5467, G_loss=1.5272, Real_S=0.63, Fake_S=0.33]
Epoch 34/50: 100%                    468/468 [00:15<00:00, 34.32it/s, D_loss=0.5546, G_loss=1.1298, Real_S=0.65, Fake_S=0.40]
Epoch 35/50: 100%                    468/468 [00:14<00:00, 31.74it/s, D_loss=0.5720, G_loss=1.1670, Real_S=0.64, Fake_S=0.38]
  -> Saved sample for epoch 35
Epoch 36/50: 100%                    468/468 [00:15<00:00, 37.01it/s, D_loss=0.5779, G_loss=1.1966, Real_S=0.64, Fake_S=0.41]
Epoch 37/50: 100%                    468/468 [00:15<00:00, 33.69it/s, D_loss=0.6122, G_loss=1.0259, Real_S=0.59, Fake_S=0.40]
Epoch 38/50: 100%                    468/468 [00:15<00:00, 26.01it/s, D_loss=0.5688, G_loss=1.0755, Real_S=0.66, Fake_S=0.44]
Epoch 39/50: 100%                    468/468 [00:16<00:00, 22.93it/s, D_loss=0.5518, G_loss=0.9635, Real_S=0.66, Fake_S=0.40]
Epoch 40/50: 100%                    468/468 [00:15<00:00, 32.45it/s, D_loss=0.5994, G_loss=1.3120, Real_S=0.56, Fake_S=0.34]
  -> Saved sample for epoch 40
```